OVAGRI Groupe
Direction des opérations · Informatique

Dossier technique
Application Cantine

Architecture, déploiement et exploitation — à l'attention de l'équipe IT / DevOps

Version du 5 novembre 2026 · code : `github.com/ahmeds1001/app-canteen` (branche `main`)
État : application développée et testée (148 tests automatiques, aussi sur SQL Server 2022), **pas encore déployée**.
Documents liés : cahier des charges (`docs/cahier-des-charges.pdf`), guide de test (`docs/guide-test-semaine.pdf`).

En une phrase

Une application web (site installable sur le téléphone) où ~200 employés réservent leur repas la veille ; la cuisine prépare réservations + 10 % ; un téléphone de la cantine scanne un QR code par repas. Elle tourne dans **trois conteneurs Docker** (site, API, tâches planifiées) derrière le proxy HTTPS de l'entreprise, avec **sa propre base SQL Server**, isolée de l'ERP.

1. Synthèse pour l'IT

Ce que l'application demande

- Un serveur **Linux** avec **Docker + Docker Compose** (Windows Server seulement s'il fait tourner des conteneurs Linux) ; **4 vCPU, 4 Go de RAM, 20 Go de disque** — estimation avec marge pour un lancement où tout le monde se connecte en même temps, à confirmer par le test de charge (§9.1).
- Une **base SQL Server 2019+** dédiée (`canteen`) et un login SQL qui n'a des droits que sur elle.
- Un **sous-domaine** (ex. `cantine.ovagri.tn`) avec certificat HTTPS sur le proxy de l'entreprise.
- Un compte **SMTP** pour 5 emails de compte (code, validation, refus, mot de passe).
- Un accès **Internet sortant en 443** vers les services push (Google, Mozilla, Apple, Microsoft).
- Une **horloge serveur juste** (NTP) : les délais de réservation se jouent à la minute.

Ce que nous attendons de vous

1. Choix du serveur et de son OS ; Docker autorisé ou non.
2. Le sous-domaine et qui gère le certificat.
3. Création de la base et du login SQL (script en §8.2).
4. Identifiants SMTP et adresse d'expéditeur.
5. Ouverture des flux réseau du §7.3, et un proxy qui transmet `X-Forwarded-For` (§7.2).
6. Sauvegardes : outil et emplacement hors du serveur ; un test de restauration avant le lancement.
7. Supervision : qui reçoit l'alerte si le site ou les tâches planifiées s'arrêtent.

Chiffres clés

Élément	Valeur
Utilisateurs	~200 employés, 7 rôles (employé, RH, chef, scanner, caissier, Direction, admin) ; un rôle par compte
Pics de charge	Les dernières minutes avant la clôture des réservations (23:59 la veille par défaut) et le service de midi (scans, 12:00–15:00)
Volume de données	Faible : ~60 000 réservations par an, quelques Mo ; plus les photos des plats (≈ 200 Ko chacune). **Rien n'est jamais supprimé** (décision de l'entreprise)
Disponibilité visée	99 % pendant le service et le soir ; une coupure de quelques minutes le reste du temps est sans conséquence
Fuseau horaire	`Africa/Tunis` pour toutes les règles ; dates stockées en UTC dans la base
Langues	Français (défaut) et anglais ; arabe prévu plus tard

2. Vue d'ensemble

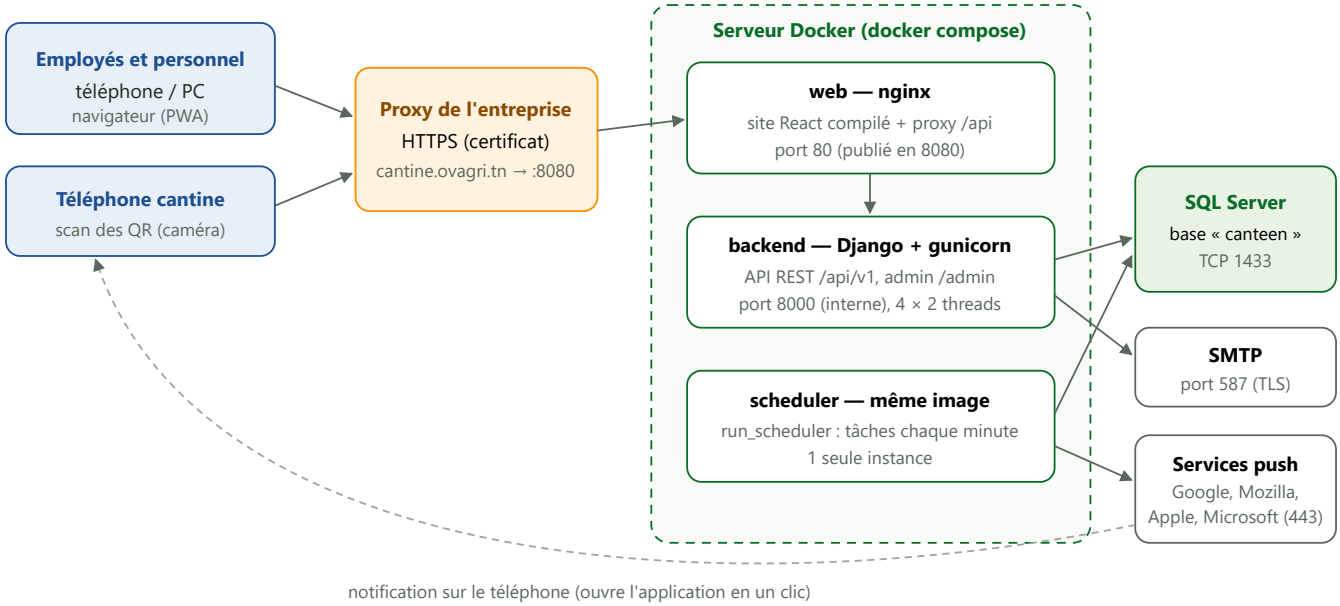


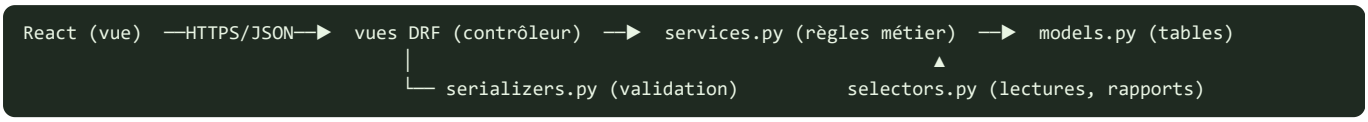
Figure 1 — Composants et flux. Seul le conteneur « web » est exposé ; l'API et les tâches parlent à SQL Server, SMTP et aux services push.

Composants

Conteneur	Technologie	Rôle
web	nginx 1.27 (alpine) ; site compilé par Vite (Node 22, à la construction seulement)	Sert le site (React 18 + TypeScript, PWA : installable, service worker, notifications push) et les photos des plats (`/media/`) ; relaie `/api/` , `/admin/` , `/static/` vers le backend
backend	Python 3.12, Django 5.2 LTS, Django REST Framework 3.18, SimpleJWT 5.5, gunicorn (4 workers × 2 threads), WhiteNoise, pilote ODBC 18, mssql-django 2.0	API REST `/api/v1/` (toutes les règles métier), site d'administration technique `/admin/` et ses fichiers statiques (WhiteNoise), envoi des emails
scheduler	Même image que backend, commande `python manage.py run_scheduler`	Tâches planifiées chaque minute (§4.3) : publication automatique, clôture, finalisation, absences, rappels, envoi des notifications push
SQL Server	SQL Server 2019+ de l'entreprise (testé sur 2022)	Base dédiée `canteen` ; aucune donnée ERP ; aucun accès de l'application à l'ERP

3. Organisation du logiciel

Architecture en couches : le site (vue) appelle l'API en JSON ; dans l'API, les vues restent minces et toutes les règles du cahier des charges sont dans les `services.py`.



Application Django	Contenu
`core`	Horloge (une seule source de l'heure, pour les tests), paramètres de la cantine (écran admin), journal d'audit, droits par rôle, tâches planifiées
`accounts`	Comptes, inscription avec code email, validation RH et matricule, mots de passe, sessions
`menus`	Bibliothèque de plats (photos), menus du jour (1 ou 2 menus), publication, clôture, finalisation du nombre d'assiettes
`reservations`	Réservations, QR codes signés, scan, listes du jour, absences
`wallet`	Solde prépayé : recharges (caissier, Direction), paiement à la réservation, remboursements, corrections
`notifications`	Notifications dans l'application et sur le téléphone (Web Push)
`reports`	Présence par employé, encaissements du jour, exports CSV

Le site (`frontend/src`) : un fichier par écran et par rôle, tous les textes dans `i18n/fr.json` et `en.json`, un seul module pour appeler l'API (`api/endpoints.ts`).

4. Fonctionnement

4.1 Sessions et API

- Connexion par email + mot de passe → jeton d'accès JWT valable **30 minutes** et jeton de renouvellement de **14 jours**, chacun utilisable une fois (rotation + liste noire). Un compte suspendu n'obtient plus de jeton, même sur un téléphone déjà connecté. Les jetons sont gardés par le navigateur (localStorage) ; pas de cookie de session pour l'API.
- Toutes les routes sont sous `/api/v1/` ; chaque route vérifie le rôle côté serveur (l'écran ne fait que masquer).
- Limitation des tentatives **par adresse de l'utilisateur** : connexion 10/min, inscription 5/h, codes 20/h, mot de passe oublié 3/h. Les compteurs sont partagés par tous les workers (table de cache dans la base).

4.2 QR code et scan

Chaque réservation porte un jeton aléatoire **signé** (clé `CANTEEN_SIGNING_KEY`), sans donnée personnelle. Le téléphone de la cantine lit le QR avec sa caméra (le navigateur exige HTTPS pour la caméra) et l'API passe la réservation à « Servi » par une mise à jour conditionnelle : deux scans simultanés ne peuvent pas réussir tous les deux.

4.3 Tâches planifiées (conteneur `scheduler`)

Une boucle exécute ces tâches **chaque minute**. Chacune est idempotente : la relancer ou la lancer en retard donne le même résultat (après une coupure, le premier passage rattrape tout).

Tâche	Effet
`publish_placeholders`	Publie « Menu à venir » si le chef n'a pas publié à temps
`lock_menus`	Clôture les réservations à l'heure limite (23:59 la veille par défaut)
`finalization_reminder`, `menu_missing_reminder`, `evening_reservation_reminder`	Rappels (Direction, chef, employés) aux heures des paramètres
`auto_finalize`	Fixe le nombre d'assiettes (réservations + marge) si la Direction ne l'a pas fait à 10:00

`mark_no_shows`, `close_finished_days`	Absences et fin de journée après le service
`send_pending_pushes`	Envoie les notifications aux téléphones (timeout 10 s, durée de vie 24 h ; un abonnement mort est retiré)

> **Une seule instance du conteneur `scheduler`.** Les tâches sont protégées contre les doublons, mais deux planificateurs enverraient deux fois plus de requêtes pour rien. Ne pas le mettre à l'échelle.

4.4 Emails et notifications

- **Email** (SMTP) seulement pour le compte : code d'inscription, compte validé ou refusé, mot de passe oublié ou changé. Envoyés après l'enregistrement en base, pour qu'un SMTP lent ne bloque rien.
- **Notifications** dans l'application et sur le téléphone (standard Web Push, clés VAPID) : rappels, menu modifié, solde rechargé… Le serveur appelle le service push du navigateur (Google FCM pour Chrome/Android, Mozilla, Apple, Microsoft) en HTTPS sortant ; sans clés VAPID, les notifications restent visibles dans l'application.

5. Données

- **Base** : SQL Server, base dédiée `canteen`, accès par le pilote ODBC Driver 18. 13 tables métier (utilisateurs, plats, menus et leurs plats, réservations, mouvements de solde, notifications et abonnements push, codes email, liens de réinitialisation, paramètres, journal d'audit) + les tables Django.
- **Schéma** : créé et modifié uniquement par les migrations Django (`python manage.py migrate`), jamais à la main. Les migrations sont ajoutées par le développeur et appliquées à chaque mise à jour.
- **Rien n'est supprimé** (décision de l'entreprise) : pas de suppression d'utilisateurs, de réservations ni de lignes d'audit ; la suppression est bloquée dans l'administration. Le journal d'audit est en ajout seulement.
- **Fichiers** : les photos des plats (`media/dishes/`, réduites à 1200 px), dans le volume Docker `media` : elles survivent aux mises à jour et sont sauvegardées avec la base (voir §9).
- **Dates** : stockées en UTC ; toutes les règles et l'affichage utilisent `Africa/Tunis`.
- **Données personnelles** : nom, email professionnel, matricule, département, repas, solde. Exports CSV tracés. Loi organique n° 2004-63 : une vérification rapide avec la RH / le juridique est recommandée.

6. Sécurité

Sujet	Mesure
Transport	HTTPS obligatoire (proxy de l'entreprise) ; en production Django redirige vers HTTPS, cookies sécurisés, HSTS 30 jours
Mots de passe	Argon2 ; au moins 8 caractères, pas courant, pas que des chiffres, pas proche du nom
Codes et liens	Code email à 6 chiffres (10 min, 5 essais) et liens de réinitialisation (30 min) stockés **hachés**
Droits	Vérifiés par l'API pour chaque route et chaque rôle (testé : chaque rôle × chaque route) ; un rôle par compte
Secrets	Uniquement dans le fichier `.env` du serveur (jamais dans Git) ; la production refuse de démarrer sans `DJANGO_SECRET_KEY`
Audit	Validations, rôles, menus, réservations, scans, recharges, corrections, paramètres, exports : qui, quoi, quand, avant/après
Administration	`/admin/` réservé aux comptes admin ; à restreindre au réseau interne sur le proxy si possible
Isolement	Conteneurs, base, login SQL et sous-domaine propres ; aucun droit sur l'ERP ; limites CPU/mémoire

7. Déploiement cible

7.1 Docker Compose (fichier `docker-compose.yml` à la racine)

Fichiers de déploiement dans le dépôt : `docker-compose.yml`, `backend/Dockerfile`, `frontend/Dockerfile`, `frontend/nginx.conf`, `.env.example`.

Service	Image	Port	Notes
migrate	image du backend	aucun	Applique les migrations de la base puis s'arrête ; backend et scheduler attendent qu'il ait réussi
backend	construite depuis `backend/Dockerfile`	8000 (interne)	Limite 2 CPU / 2 Go ; 4 workers × 2 threads (`backend/gunicorn.conf.py`) ; **healthcheck** toutes les 30 s sur `/api/v1/health/` (vérifie aussi SQL Server) ; volume `media`
scheduler	image du backend	aucun	**Une seule instance**
web	construite depuis `frontend/Dockerfile`	80 → **8080** sur l'hôte	Le seul port publié ; démarre quand le backend est en bonne santé ; lit le volume `media`

Tous les services redémarrent seuls (`restart: unless-stopped`) et leurs journaux tournent (10 Mo × 5 fichiers).

7.2 Ce que le proxy HTTPS doit faire

- Terminer TLS pour `cantine.<domaine>` et relayer vers `http://<serveur>:8080`.
- Transmettre les en-têtes `Host` et `X-Forwarded-Proto: https` (indispensable : Django s'en sert pour savoir que la requête était en HTTPS ; le conteneur web le transmet tel quel).
- **Ajouter l'adresse de l'utilisateur dans `X-Forwarded-For`** (indispensable aussi) : la limite de 10 connexions par minute compte par adresse. Sans cet en-tête, tous les employés auraient la même adresse et **seules 10 personnes par minute pourraient se connecter dans toute l'entreprise**. Si un autre proxy s'ajoute devant, augmenter `NUM_PROXIES`.
- Taille de requête ≥ 6 Mo (photos des plats jusqu'à 5 Mo). Pas de cache sur `/api/`.

7.3 Flux réseau

Source	Destination	Port	Usage
Utilisateurs (Internet / Wi-Fi)	Proxy de l'entreprise	443	Le site et l'API
Proxy	Serveur Docker	8080	Conteneur web
backend, scheduler	SQL Server	1433	Base `canteen`
backend	Serveur SMTP	587	Emails de compte
scheduler	fcm.googleapis.com, updates.push.services.mozilla.com, *.push.apple.com, *.notify.windows.com	443	Notifications sur les téléphones
Hôte Docker (construction)	Docker Hub, PyPI, npm, packages.microsoft.com	443	Construire les images (pas nécessaire à l'exécution)

Le téléphone de la cantine doit avoir un réseau fiable dans la salle (Wi-Fi ou 4G) : à vérifier avant le lancement.

7.4 Variables d'environnement (fichier `.env` à côté de `docker-compose.yml`)

Variable	Rôle et valeur en production
`DJANGO_SECRET_KEY` secret	Chaîne aléatoire longue (50+ caractères)
`CANTEEN_SIGNING_KEY` secret	Autre chaîne aléatoire ; signe les QR et hache les codes. **La changer invalide les QR existants**

`DJANGO_DEBUG`	`false`
`DJANGO_ALLOWED_HOSTS`	`cantine.ovagri.tn` (le sous-domaine)
`CSRF_TRUSTED_ORIGINS`, `CORS_ALLOWED_ORIGINS`	`https://cantine.ovagri.tn`
`FRONTEND_URL`	`https://cantine.ovagri.tn` (liens dans les emails)
`COMPANY_TIME_ZONE`	`Africa/Tunis`
`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` **secret**	Serveur SQL, 1433, `canteen`, `canteen_app`, mot de passe du login
`DB_EXTRA_PARAMS`	`TrustServerCertificate=yes` par défaut ; à remplacer par `Encrypt=yes` si le serveur SQL a un certificat reconnu
`EMAIL_HOST`, `EMAIL_PORT`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD` **secret**, `EMAIL_USE_TLS`, `DEFAULT_FROM_EMAIL`	Compte SMTP de l'entreprise ; expéditeur ex. `Cantine OVAGRI <cantine@ovagri.tn>`
`VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY` **secret**, `VAPID_SUBJECT`	Clés des notifications push, générées une fois (§8.3) ; vides = pas de push. Les changer oblige chaque téléphone à réactiver les notifications
`NUM_PROXIES`	`2` : nombre de proxys devant Django (proxy de l'entreprise + nginx), pour retrouver l'adresse de l'utilisateur
`GUNICORN_WORKERS`, `GUNICORN_THREADS`	`4` et `2` : requêtes traitées en même temps ; à augmenter avec la limite mémoire du backend
`DB_CONN_MAX_AGE`	`60` : secondes de réutilisation d'une connexion SQL Server (vérifiée avant usage)
`THROTTLE_LOGIN`	Ne pas définir en production (10/min par défaut). Seulement pour le test de charge (§9.1)
`ENV_FILE`, `WEB_PORT`	Variables du shell (pas du `.env`) pour lancer une 2e installation de test à côté : autre fichier de réglages, autre port
`LOG_LEVEL`	`INFO` (par défaut)

8. Installation et mises à jour

8.1 Première installation

```
git clone https://github.com/ahmeds1001/app-canteen.git cantine && cd cantine
cp .env.example .env            # puis remplir les valeurs du §7.4
docker compose up -d --build    # construit, applique les migrations, démarre
docker compose run --rm backend python manage.py createsuperuser   # le premier compte admin
```

8.2 Base de données (à faire par l'administrateur SQL Server)

```
CREATE DATABASE canteen COLLATE French_CI_AS;
CREATE LOGIN canteen_app WITH PASSWORD = '<mot de passe fort>';
USE canteen;
CREATE USER canteen_app FOR LOGIN canteen_app;
ALTER ROLE db_owner ADD MEMBER canteen_app;   -- sur cette base seulement (les migrations créent les tables)
```

8.3 Clés des notifications push (une fois)

```
docker compose run --rm backend python manage.py generate_vapid_keys   # 2 lignes à copier dans .env
docker compose up -d                                                    # pour les prendre en compte
```

8.4 Mise à jour

```
git pull
docker compose up -d --build    # les migrations passent avant le redémarrage de l'API
```

Moins d'une minute d'interruption ; à faire hors des pics (pas entre 23:30 et 00:15, ni entre 11:45 et 15:00). Les tâches planifiées rattrapent seules ce qui est tombé pendant la coupure.

8.5 Vérifications après installation

- `https://cantine.<domaine>/api/v1/health/` répond `{"status": "ok", "database": "ok", "server_time": …}` avec l'heure de Tunis ; `docker compose ps` montre le backend « healthy ».
- `docker compose logs scheduler` montre une ligne « Scheduler tick » par minute, sans `-1` (tâche en échec).
- Inscription d'un compte de test : l'email avec le code arrive.
- Sur un téléphone : « Activer » les notifications, recharger un solde, la notification arrive.
- `/admin/` s'affiche avec sa mise en forme ; une photo de plat ajoutée par le chef s'affiche sur l'accueil.

9. Exploitation

Sujet	Consigne
Sauvegardes	Sauvegarde **complète quotidienne** de la base `canteen` (+ journaux de transactions si mode de récupération complet) et du volume Docker des photos (`<projet>_media`), copiées **hors du serveur**. Garder au moins 30 jours. **Un test de restauration avant le lancement**, puis chaque trimestre
Journaux	Sortie standard des conteneurs (`docker compose logs`), avec rotation automatique (10 Mo × 5 fichiers par service)
Supervision	Alerte si `/api/v1/health/` ne répond pas 200 (API ou base indisponible), ou si le conteneur `scheduler` s'arrête (sinon : pas de clôture, pas de finalisation automatique, pas de rappels)
Heure	NTP actif sur l'hôte Docker. Une dérive de quelques minutes décale la clôture des réservations
Paramètres métier	Horaires, délais, marge, prix : écran « Paramètres » de l'application (compte admin), pas de redéploiement
Charge	Test de charge avant le lancement (§9.1)
Comptes	Le premier admin est créé à l'installation (§8.1) ; ensuite les rôles se donnent dans l'écran « Comptes »

9.1 Test de charge (avant le lancement)

Le kit est dans le dépôt : `loadtest/` (mode d'emploi `loadtest/README.md`). Il se lance sur une **installation de test à côté de la vraie**, avec sa propre base `canteen_loadtest` : les 200 comptes de test ne doivent jamais entrer dans la base réelle (rien n'y est supprimé). La commande `loadtest_accounts` crée les comptes avec 100 TND et ouvre un jour à deux menus ; elle refuse de tourner sur une base dont le nom ne contient pas « loadtest ».

Scénario	Ce qu'il simule	Réussi si (95 % des requêtes)
`LoginRush`	Jour du lancement : 200 comptes se connectent une fois, dans la même minute	connexion < 5 s, moins de 1 % d'erreurs
`ReservationRush`	Rush du soir : 200 personnes pendant 5 min (menus, réserver, changer de menu, annuler, solde)	< 1 s (connexion < 5 s), moins de 1 % d'erreurs

Essai à blanc sur le poste de développement (serveur de développement, un seul processus) : 60 connexions simultanées et 1 027 requêtes de rush, aucune erreur, 95 % des réservations en moins de 63 ms. Il valide le kit ; le vrai chiffre viendra du serveur. La connexion est volontairement la requête la plus lourde (vérification Argon2 du mot de passe, ~100 Mo) : c'est elle qui dimensionne le CPU et la mémoire du backend.

10. État de la configuration Docker

Cinq défauts relevés en préparant ce document ont été **corrigés le 5 novembre 2026** :

#	Défaut	Correction
1	Mise en forme de `/admin/` absente (fichiers statiques non servis sans `DEBUG`)	WhiteNoise dans le backend ; fichiers compressés et versionnés à la construction de l'image
2	Photos des plats non affichées et perdues à chaque reconstruction	Volume `media` partagé ; nginx sert `/media/`
3	Boucle de redirection HTTPS derrière le proxy	nginx transmet le `X-Forwarded-Proto` reçu du proxy
4	Migrations et contrôle de santé manuels	Service `migrate` automatique ; healthcheck du backend (API + base)
5	Photos de plus de 1 Mo refusées par nginx	`client_max_body_size 6m`

À valider lors de la première installation. Les corrections 1, 3 et 4 côté Django sont couvertes par des tests automatiques et vérifiées avec les réglages de production ; la configuration nginx et Docker Compose n'a pas encore tourné dans Docker (pas de Docker sur le poste de développement). Les vérifications du §8.5 le confirment.

Calendrier proposé

1. IT : serveur, base SQL, sous-domaine, SMTP, flux réseau (§1).
2. Installation avec l'IT (§8) et vérifications (§8.5).
3. Sauvegarde + test de restauration ; test de charge (§9.1) ; ouverture des inscriptions quelques jours avant, service par service, pour étaler les premières connexions.
4. Pilote avec un service pendant une à deux semaines, puis ouverture à tous.